VASP Workflow

User Manual

Browser GUI · CLI Agents · SLURM HPC

Keivan Esfarjani

June 2026 | https://github.com/KeivanS/VASP-Flow

Contents

1 Introduction

VASP Workflow is a framework for setting up, running, and analysing DFT calculations with VASP. Two interfaces are provided:

- **Browser GUI** (`vasp-gui.py`) — a Flask web application: configure calculations through a form, run steps from the browser, view band-structure and DOS plots in real time.
- **CLI agents** (`vasp-agent.py` for workstations, `vasp-agent-slurm.py` for SLURM clusters) — read a plain-text `instructions.txt` file and generate all INCAR, KPOINTS, POSCAR, POTCAR, and job scripts automatically.

Both interfaces share the same `modules/` back-end.

1.1 Key features

- Functionals: PBEsol, PBE, R2SCAN, HSE06, RVV10, LDA, AM05
- Spin-orbit coupling (SOC) and collinear spin (ISPIN=2)
- Magnetic materials: correct MAGMOM per atom; spin-resolved band and DOS plots (spin-up / spin-down)
- GGA+U (Dudarev, LDAUTYPE=2) per element and orbital
- Automatic k-path detection from POSCAR lattice vectors
- Convergence testing: ENCUT and k-mesh sweeps
- DOS with orbital projections via `sumo`
- Phonons via phonopy, Wannier90 interface, DFPT Born charges
- Smart KPAR/NCORE auto-defaults scaled with MPI task count
- SLURM workflow with dependency-chained job submission

1.2 Requirements

Package	Notes
VASP 5.4+	`vasp_std`, `vasp_ncl` (SOC), optionally `vasp_gam`
POTCAR library	PAW pseudopotentials in per-element folders
Python 3.8+	Conda environment recommended
Flask	`pip install flask` — GUI only
sumo $\geq$ 2.0	`pip install sumo` — band and DOS plots
MPI	`mpirun`, `mpiexec`, or `srun`
phonopy	optional; `conda install -c conda-forge phonopy`
wannier90.x	optional; `conda install -c conda-forge wannier90`

2 Installation

```
# 1. Clone the repository
git clone https://github.com/KeivanS/VASP-Flow.git
cd VASP-Flow

# 2. Install Python dependencies
pip install flask sumo

# 3. Optional: phonons and Wannier support
conda install -c conda-forge phonopy wannier90

# 4. First-time configuration (creates site.env from template)
make setup
nano site.env      # set VASP paths, POTCAR directory, MPI command
```

2.1 site.env — platform configuration

```
PYTHON          = python3
VASP_STD        = /path/to/vasp_std
VASP_NCL        = /path/to/vasp_ncl
VASP_GAM        = /path/to/vasp_gam       # optional
VASP_POTCAR_DIR = /path/to/potpaw_PBE
MPI_LAUNCH      = mpirun -np              # or: srun
MPI_NP          = 16
WANNIER90_X     = wannier90.x
```

3 Browser GUI

3.1 Starting the GUI

```
make run           # launches on http://localhost:5001
```

On a SLURM cluster — run on the login node, open on your laptop via SSH port-forwarding:

```
# On your laptop:
ssh -L 5001:localhost:5001 username@cluster

# On the cluster login node:
make run

# Open http://localhost:5001 in your laptop's browser
```

3.2 Three-tab workflow

1. **Setup** — configure the project, then click *Generate Workflow*.
2. **Workflow** — run each calculation step; live VASP output streams to the browser.
3. **Results** — band-structure and DOS plots; OUTCAR analysis.

To return to a previous project use the *Select project* dropdown. → **Workflow** jumps to the run tab; **Edit & Regenerate** re-opens Setup with all settings pre-filled.

3.3 Setup tab — field guide

3.3.1 Structure and POTCAR

Field	Description
Project name	Folder created in the working directory.
POSCAR	Paste the crystal structure, or click *Load file*.
POTCAR directory	Path to your PAW library. If an element has multiple variants (`Ga` vs `Ga_d`) a chooser appears.

3.3.2 Functional

Choice	Description
PBEsol	Optimised for solids; recommended starting point.
PBE	Standard GGA; widely used.
R2SCAN	Meta-GGA; good accuracy/cost balance.
HSE06	Hybrid — accurate band gaps, significantly slower.
RVV10	van der Waals; for layered or molecular materials.
LDA / AM05	Rarely needed.

3.3.3 Spin / SOC

Choice	Effect
None	Non-magnetic.
Collinear spin (ISPIN=2)	Sets `ISPIN = 2`; reveals the *Magnetic moment / atom* field (Section 7.3).
SOC ∥ z/x/y	Sets `LSORBIT`, `LNONCOLLINEAR`, `SAXIS`; uses the `vasp_ncl` binary.

3.3.4 Other flags

Field	Effect
Hexagonal BZ	Correct M point for hexagonal structures.
2D / slab	Sets $k_z = 1$ (Gamma-only out-of-plane).
GGA+U	Adds Hubbard U rows per element/orbital (Section 7.4).
Execution profile	Select *SLURM HPC* to generate `sbatch` scripts; edit `profiles/slurm.json` first.
MPI tasks	CPU cores per calculation.

3.4 Task checkboxes

Task	Directory	Notes
Structure relaxation	`01_relax`	IBRION=2; run first if geometry is not pre-optimised.
SCF	`02_scf`	Required before bands or DOS.
Band structure	`03_bands`	Line-mode KPOINTS; Section 9.
DOS	`04_dos`	Dense k-mesh; orbital projections via sumo.
DFPT	`06_dfpt`	Born charges + dielectric (IBRION=8).
Phonons	`07_phonons`	Phonopy finite-displacement; DFPT needed for NAC.
Wannier90	`05_wannier`	NSCF interface; Section 14.

Note. Normal execution order: **relaxation** → **SCF** → **bands** → **DOS**. Each step copies POSCAR/CHGCAR from the previous one automatically.

3.5 Band structure options

Leave the *k-path* field blank to trigger **automatic detection** from the POSCAR lattice vectors (Section 9). Enter a path manually to override, e.g. `G-M-K-G`.

3.6 DOS options

Add projection rows to request orbital-resolved DOS:

Element: `Mo` Orbitals: **d** Element: `S` Orbitals: **p**

For spin-polarised calculations sumo plots spin-up positive and spin-down mirrored below zero automatically.

3.7 Workflow tab

Each step shows three controls: **Run** (submit the step), **Edit files** (open INCAR/KPOINTS/POSCAR in the browser; a `.bak` is created on first edit), and a **status badge**. **Run All Steps** chains every step and stops on failure.

3.8 Results tab

Panel	Contents
Band structure	Spin-polarised: blue = ↑, red dashed = ↓, same k-axis, $E_F = 0$. Non-magnetic: single-colour sumo plot.
Cumulative DOS	Stacked filled-area by element. Spin-polarised: up positive, down mirrored.
Projected DOS	Orbital-resolved via sumo; spin-down negated automatically.
OUTCAR analysis	Energy, Fermi level, magnetic moment, forces.

4 CLI Agents

4.1 vasp-agent.py — workstation

```
./vasp-agent.py
./vasp-agent.py -i my_inst.txt -s struct.vasp

cd ProjectName
./run_all.sh          # runs every step in sequence
./analyze.sh          # plots after all steps complete
```

4.2 vasp-agent-slurm.py — SLURM clusters

```
./vasp-agent-slurm.py                  # uses profiles/slurm.json
./vasp-agent-slurm.py --profile slurm

cd ProjectName
./submit_all.sh        # sbatch with dependency chaining
squeue -u $USER        # monitor jobs
./analyze.sh           # run locally after jobs finish
```

Two-phase workflow when convergence tests are requested:

```
./submit_convergence.sh     # STEP 1 -- submit convergence jobs
# review results, then:
./submit_calculations.sh    # STEP 2 -- prompts ENCUT/k-mesh, submits
```

4.3 Generated directory structure

```
ProjectName/
  instructions.txt
  POSCAR
  POTCAR                built once from $VASP_POTCAR_DIR
  01_relax/             INCAR  KPOINTS  POSCAR  POTCAR  run.sh
  02_scf/
  03_bands/
  04_dos/
  analysis/             plots generated by analyze.sh
  run_all.sh            workstation sequential runner
  submit_all.sh         SLURM job submission (dependency chained)
  analyze.sh            post-processing: plots + OUTCAR extraction
```

5 High-Throughput Screening (Materials Project)

`ht-mp-scf.py` drives a batch of SCF + ELF calculations straight from Materials Project IDs. For every `mp-id` it downloads the **primitive** cell from MP, stages a per-material `instructions.txt` (an SCF task plus an `INCAR scf:` block setting `LELF = .TRUE.` for the electron localization function), and writes a `runall.sh` that calls the chosen agent to build the inputs and then runs or submits each job. Each SCF uses a medium `10 10 10` Γ-mesh by default; edit the `KMESH` line in `_ht_inputs/<id>/instructions.txt` (or pass `--kmesh`) to change it.

```
pip install pymatgen mp-api          # MP download + primitive
    standardisation
export MP_API_KEY=...                 # Materials Project API key
export VASP_POTCAR_DIR=...            # POTCAR library (used by the
    agent)

# one mp-id per line (blank lines and # comments ignored)
printf 'mp-149\nmp-2534\n' > highthrouput_list

./ht-mp-scf.py --agent local  --mpi 16        # workstation
./ht-mp-scf.py --agent slurm  --profile slurm # cluster (chained by
    default)

./runall.sh                           # build inputs + run/submit
    every job
```

The ELFCAR for each material is written to `<mp-id>/02_scf/`. A summary `_ht_inputs/elements_Z.txt` tabulates every element in the screening set with its atomic number Z (from a built-in table — no MP query), plus a per-material breakdown.

Option	Meaning
`--agent`	`local` (vasp-agent.py) or `slurm` (vasp-agent-slurm.py); prompts if omitted.
`--list`	ID file; default `highthrouput_list`.
`--functional`	PBE (default), PBEsol, R2SCAN, …
`--mpi`	MPI tasks per job.
`--encut`	Optional ENCUT override (eV).
`--kmesh`	SCF Γ-mesh, e.g. `'8 8 8'` or `'12'` (cubic); default medium `10 10 10`. Per-material edits via the `KMESH` line in `instructions.txt`.
`--chain` / `--no-chain`	SLURM only; see below.

5.1 Execution order

Mode	Behaviour
local	`run_all.sh` blocks — materials run strictly one-at-a-time.
slurm `--chain` (default)	Each material's SCF is submitted with `--dependency=afterok` on the previous one, so the cluster runs them sequentially even though all are queued up front.
slurm `--no-chain`	Materials are submitted independently and run whenever the scheduler allows.

Note. The driver fetches structures over the network (Materials Project), so run it on a node with internet — typically the login node. `runall.sh` performs no downloads and can run on a compute node.

6 instructions.txt Format

6.1 Example

```
Project: My material name

Methods: PBEsol functional
         collinear magnetization
         GGA+U with U=4.0 on Fe-d orbitals

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Fe-d and O-p)

Convergence: Test k-points 6x6x6 to 12x12x12
             ENCUT 400-600 eV

MAGMOM: 4.0
NSW: 100
EDIFFG: -0.01
MPI: 128
NODES: 2
NTASKS_PER_NODE: 64
WALLTIME: 12:00:00
```

The parser is case-insensitive and searches the entire file, so section order is flexible.

6.2 Keyword reference

Keyword	Meaning
`Project:`	Project name; becomes the output folder.
`MAGMOM:`	Initial moment in μ_B/atom (collinear only). Written as `MAGMOM = N*value`. Default 2.0.
`ISIF:`	Relaxation type: 3=full, 2=ions only, 4=fixed volume.
`NSW:`	Max ionic steps. Default 100.
`EDIFFG:`	Force criterion in eV/Å. Default −0.01.
`ENCUT:`	Manual plane-wave cutoff in eV.
`KMESH:`	Explicit Gamma k-mesh overriding the automatic density, e.g. `8 8 8` or `12` (cubic). Applies to every automatic mesh (relax/SCF/DOS); for 2-D give the third value (`12 12 1`).
`PRESSURE:`	External pressure for constant-P relaxation, e.g. `10 GPa` or `50 kbar`. Forces `ISIF=3`, `IBRION=2`, `PSTRESS` (Section 8.2).
`INCAR: ... END_INCAR`	Block of literal INCAR tags merged into the generated INCAR(s); optionally per-step (Section 8.1).
`NKPTS:`	k-points per band segment. Default 40.
`MPI:`	Total MPI tasks (nodes × cores/node).
`KPAR:`	Global k-point parallelisation.
`NCORE:`	Global cores-per-orbital.
`SCF_KPAR:` *etc.*	Per-step KPAR: also BANDS_, DOS_, RELAX_, DFPT_, PHONONS_.
`SCF_NCORE:` *etc.*	Per-step NCORE; same prefixes.
`NODES:`	Compute nodes (SLURM).
`NTASKS_PER_NODE:`	MPI tasks per node (SLURM).
`PARTITION:`	SLURM partition.
`WALLTIME:`	Time limit, e.g. `12:00:00`.
`ACCOUNT:`	SLURM account / allocation.
`DFPT_EDIFF:`	EDIFF for DFPT. Default `1E-8`.
`PHONONS_DIM:`	Supercell, e.g. `2 2 2`.
`PHONONS_MESH:`	q-mesh, e.g. `20 20 20`.
`PHONONS_DISP:`	Displacement in Å. Default 0.01.
`PHONONS_BAND:`	High-symmetry path for phonon bands.
`PHONONS_NAC:`	`FALSE` to disable NAC.
`WANNIER_NUM_WANN:`	Number of Wannier functions.
`WANNIER_PROJ:`	Projections, e.g. `Ga:s;As:p`.
`WANNIER_EWIN:`	Energy window, e.g. `-5.0 to 15.0`.

7 Methods

7.1 Functionals

```
Methods: PBEsol functional     # recommended for solids
Methods: R2SCAN functional     # meta-GGA
Methods: HSE06 functional      # hybrid -- more accurate but slower
Methods: RVV10 functional      # van der Waals
```

Default ENCUT values: PBE 400 eV, PBEsol 450 eV, R2SCAN 680 eV, HSE06 400 eV, RVV10 450 eV.

7.2 Spin-orbit coupling

```
Methods: SOC with magnetization in z-direction
```

Uses `vasp_ncl`. Sets `LSORBIT`, `LNONCOLLINEAR`, and `SAXIS`.

7.3 Magnetic materials (collinear spin)

```
Methods: collinear magnetization
MAGMOM: 4.0
```

Sets `ISPIN = 2` and `MAGMOM = N * value` in every INCAR, where N is the atom count from the POSCAR.

Tip. Typical starting moments: Fe, Co $\approx$ 4–5 μ_B; Ni $\approx$ 2 μ_B; Cr, Mn $\approx$ 3–4 μ_B; rare earths $\approx$ 7 μ_B. VASP converges the self-consistent value automatically.

Antiferromagnets — after generation, edit INCAR to alternate signs:

```
MAGMOM = 4.0 -4.0 4.0 -4.0    ! alternating per sublattice
```

Spin-resolved plots produced automatically after DOS/bands run:

- *Cumulative DOS* — spin-up above the x-axis; spin-down mirrored below zero; elements colour-coded; solid = ↑, dashed = ↓.
- *Band structure* — spin-up in blue, spin-down in red dashed, same k-axis, $E_F = 0$.
- *Projected DOS* — sumo natively negates the spin-down channel in the orbital-resolved PDF.

Check the converged moment:

```
grep "magnetization␣(x)" OUTCAR | tail -5
```

7.4 GGA+U

```
Methods: GGA+U with U=4.0 on Fe-d orbitals
         GGA+U with U=3.0 on Ni-d orbitals
         GGA+U with U=6.0 on Ce-f orbitals
```

Dudarev scheme (LDAUTYPE=2); $J = 0$ is set automatically. Combine freely with `collinear magnetization`.

8 Advanced INCAR Control

Two mechanisms let you go beyond the built-in keywords without editing each generated INCAR by hand: **raw INCAR passthrough** (inject any VASP tag directly) and **constant-pressure relaxation**.

8.1 Raw INCAR passthrough

Place literal INCAR lines — in ordinary VASP syntax — inside an `INCAR: ... END_INCAR` block in your `instructions.txt`. They are merged into the generated INCAR(s):

```
INCAR:                    # global block -- applies to every step
   LREAL   = Auto
   ALGO    = Fast
```

```
   NELMIN = 6
END_INCAR
```

A tag that the agent already writes is **overwritten in place** (so `ALGO = Fast` above replaces the default `ALGO = Normal`); any tag the agent does not normally emit is appended under a clearly labelled `# User INCAR overrides` section, so nothing is silently lost.

To target a single step, name it on the header line. Recognised steps are `relax`, `scf`, `bands`, `dos`, `wannier`, `dfpt`, `phonons`:

```
INCAR scf:              # applies only to 02_scf
   EDIFF = 1E-8
   SIGMA = 0.10
END_INCAR
```

Note. Precedence: a *per-step* block beats the *global* block, and both beat the agent's generated defaults. This is the recommended way to set tags the parser has no dedicated keyword for (e.g. `LMAXMIX`, `ADDGRID`, `LVTOT`, `IVDW`, `AMIX`).

8.2 Constant-pressure relaxation

A single `PRESSURE` line turns the relaxation into a constant-pressure optimisation: the agent forces `ISIF = 3` (cell shape + volume + ions) and `IBRION = 2`, and imposes the target pressure through `PSTRESS`.

```
PRESSURE: 10 GPa        # also accepted: "5 kbar", "constant
   pressure 8"
```

VASP's `PSTRESS` tag is in **kBar** (1 GPa = 10 kBar). The agent converts automatically; if no unit is given, **GPa is assumed**. The resulting `01_relax/INCAR` reads, e.g.:

```
# Constant-pressure relaxation @ 10 GPa  (PSTRESS = 100 kBar)
IBRION = 2
ISIF   = 3
NSW    = 100
EDIFFG = -0.01
PSTRESS = 100   ! external pressure in kBar (10 GPa)
```

Tip. `PSTRESS` adds the PV term so the relaxed cell sits at the requested external pressure. Use a tight `EDIFFG` and a well-converged `ENCUT`; the Pulay stress error grows at high pressure, so verify with a higher `ENCUT` before trusting the equilibrium volume.

9 K-Path for Band Structure

9.1 Auto-detection (recommended)

Omit the path in `instructions.txt` or leave the field blank in the GUI:

```
Tasks: band structure       # no "along ..." -> auto-detect from
   POSCAR
```

The generator reads POSCAR lattice vectors, computes cell lengths and angles, and selects the standard path.

Type	Detection criterion	Default path
cF (FCC)	a=b=c, angles ≈ 60°	G-X-W-K-G-L-U-W
cI (BCC)	a=b=c, angles ≈ 109.5°	G-H-N-G-P-H
cP (cubic)	a=b=c, 90°	G-X-M-G-R-X
hP (hexagonal)	a≈b, γ=120°	G-M-K-G-A-L-H-A
tP (tetragonal)	a≈b≠c, 90°	G-X-M-G-Z-R-A-Z
oP (orthorhombic)	90°, a≠b≠c	G-X-S-Y-G-Z-U-R-T-Z
rP (rhombohedral)	a=b=c, equal angles	G-T-F-G-L
mP (monoclinic)	α=γ=90°, β≠90°	G-Y-A-Z-G-B-D

Note. For **conventional** FCC or BCC cells (right angles, 4 or 2 atoms), auto-detection returns the cubic (cP) path. Use the **primitive** cell for an unfolded FCC/BCC band structure — the primitive FCC cell has angles ≈ 60° and is recognised automatically.

The detected type appears in the KPOINTS comment:

```
Line-mode KPOINTS  [FCC -- auto-detected]
```

9.2 Manual override

```
Tasks: band structure along G-X-W-K-G-L
```

Label	Coords	Use	
G	(0, 0, 0)	all	Γ
M	(0.5, 0.5, 0)	cubic, tet	(0.5, 0, 0) in hexagonal
K	(1/3, 1/3, 0)	hexagonal	
R	(0.5, 0.5, 0.5)	cubic	
Z	(0, 0, 0.5)	general	
A	(0, 0, 0.5)	hex, tet	
L	(0.5, 0, 0.5)	hexagonal	
H	(1/3, 1/3, 0.5)	hexagonal	
S	(0.5, 0.5, 0)	orthorhombic	
W	(0.5, 0.25, 0.75)	**FCC primitive**	
P	(0.25, 0.25, 0.25)	**BCC primitive**	
N	(0, 0, 0.5)	**BCC primitive**	

10 DOS Projections

All of the following forms are accepted:

```
DOS (projected on Mo-d and S-p)
DOS with projections on Ni:d and Ni:s
DOS with projection on Fe-d and O-p
```

Orbital channels: `s`, `p`, `d`, `f`. Separator is either `-` or `:`. Sumo generates an orbital-resolved PDF in `analysis/`.

11 Convergence Tests

```
# Range (auto-steps)
Convergence: Test k-points from 6x6x1 to 18x18x1    # 2D material
             Test k-points from 6x6x6 to 12x12x12   # 3D material
             ENCUT 400-600 eV
```

```
# Explicit mesh list
Convergence: Test k-points 6x6x3, 12x12x6, 18x18x9
```

Results in `00_convergence/kpoints/` and `00_convergence/encut/`.

```
./run_convergence.sh          # workstation
./submit_convergence.sh       # SLURM
./analyze_convergence.sh      # plots: energy / pressure / forces
```

12 Parallelization

12.1 Auto-defaults

When KPAR/NCORE are not specified the agent computes from total tasks N:

Step	KPAR	NCORE	Notes
SCF / DOS / relax	$\lfloor\sqrt{N}\rfloor$	N/KPAR	
bands	1	N	Required for line-mode
DFPT	1	N	Required for IBRION=8
phonons	1	$\lfloor\sqrt{N}\rfloor$	

Note. KPAR × NCORE must divide N evenly. The agent rounds down automatically to the nearest valid value.

12.2 Manual overrides

```
MPI: 128        KPAR: 4        NCORE: 8

# Per-step (take priority over globals)
SCF_KPAR: 4     BANDS_KPAR: 1     DOS_KPAR: 8
SCF_NCORE: 8    BANDS_NCORE: 16   DOS_NCORE: 8
RELAX_KPAR: 2   DFPT_KPAR: 1      PHONONS_KPAR: 1
```

13 SLURM Cluster Settings

13.1 profiles/slurm.json

```
{
  "vasp_std": "vasp_std",  "vasp_ncl": "vasp_ncl",
  "mpi_cmd":  "srun",
  "modules":  ["intel/2021", "impi/2021", "vasp/6.3.2"],
  "slurm": {
    "partition":       "standard",
    "nodes":           2,
    "ntasks_per_node": 64,
    "time":            "12:00:00",
    "account":         "mygroup"
  }
}
```

13.2 Per-project overrides (in instructions.txt)

```
NODES: 4      NTASKS_PER_NODE: 32      PARTITION: gpu
WALLTIME: 48:00:00      ACCOUNT: myallocation
```

14 Wannier90

```
Tasks: Wannier90 wannierization

WANNIER_NUM_WANN: 8
WANNIER_PROJ: Ga:s;As:p
WANNIER_EWIN: -5.0 to 15.0
```

The agent generates `05_wannier/wannier90.win` with the NSCF k-mesh and projection block. After the NSCF run, execute `wannier90.x` from the `05_wannier/` directory.

15 Phonons (phonopy)

```
Tasks: phonons lattice dynamics

PHONONS_DIM: 2 2 2
PHONONS_MESH: 20 20 20
PHONONS_DISP: 0.01
PHONONS_BAND: G M K G A
PHONONS_NAC: FALSE      # disable non-analytic correction
```

Note. For the non-analytic correction (important for polar materials), include `DFPT` in your task list so Born effective charges are computed.

16 Worked Examples

16.1 MoS_2 monolayer with SOC

```
Project: MoS2 monolayer

Methods: PBEsol functional
         SOC with magnetization in z-direction

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Mo-d and S-p)

Convergence: Test k-points 8x8 to 20x20
             ENCUT 400-600 eV

MPI: 64
```

The generator detects hexagonal (hP) and uses G-M-K-G-A-L-H-A.

16.2 Nickel: magnetism + GGA+U

```
Project: Nickel GGA+U

Methods: PBEsol functional
         collinear magnetization
```

```
          GGA+U with U=4.0 on Ni-d orbitals

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Ni-d and Ni-s)

MAGMOM: 2.0
MPI: 128
NODES: 2
NTASKS_PER_NODE: 64
WALLTIME: 08:00:00
```

With a primitive FCC POSCAR (angles $\approx 60°$) the generator detects cF and uses G-X-W-K-G-L-U-W. Band and DOS plots show both spin channels.

16.3 GaAs: phonons with NAC

```
Project: GaAs phonons

Methods: PBEsol functional

Tasks: SCF calculation
       DFPT Born charges and dielectric
       phonons lattice dynamics

PHONONS_DIM: 4 4 4
PHONONS_MESH: 20 20 20
PHONONS_BAND: G X W L G K

MPI: 64     WALLTIME: 24:00:00
```

16.4 High-pressure relaxation with custom INCAR tags

A constant-pressure cell relaxation at 10 GPa, combined with raw INCAR passthrough to tighten convergence and add tags the parser has no dedicated keyword for:

```
Project: Silicon 10GPa

Methods: PBEsol functional

Tasks: structure relaxation
       SCF calculation
       band structure

PRESSURE: 10 GPa          # constant-P relax -> ISIF=3, IBRION=2,
   PSTRESS=100

# Tags applied to every step
INCAR:
   LREAL  = .FALSE.
   ADDGRID = .TRUE.
   LMAXMIX = 4
END_INCAR

# Tighter electronic convergence for the relaxation only
INCAR relax:
```

```
   EDIFF  = 1E-8
   EDIFFG = -0.005
END_INCAR

ENCUT: 600
MPI: 64
```

The `01_relax/INCAR` is generated with `ISIF=3`, `PSTRESS=100`, the global tags appended, and `EDIFF`/`EDIFFG` overridden to the per-step values; `02_scf` and `03_bands` receive only the global block.

16.5 Bi_2Se_3: topological insulator

```
Project: Bi2Se3 topological

Methods: PBEsol functional
         SOC with magnetization in z-direction

Tasks: structure relaxation
       SCF calculation
       band structure
       DOS (projected on Bi-p and Se-p)
       Wannier90 wannierization

Convergence: Test k-points from 8x8x4 to 16x16x8
             ENCUT 400-600 eV

WANNIER_NUM_WANN: 18
WANNIER_PROJ: Bi:p;Se:p
WANNIER_EWIN: -2.0 to 2.0
MPI: 128
```

17 Common OUTCAR Checks

```
grep "reached required accuracy"   OUTCAR          # converged?
grep "energy  without"             OUTCAR | tail -1 # total energy
grep "E-fermi"                     OUTCAR | tail -1 # Fermi level
grep "magnetization (x)"           OUTCAR | tail -5 # moment
grep "TOTAL-FORCE"                 OUTCAR | tail -20
grep -A3 "BORN EFFECTIVE CHARGES"  OUTCAR | head -20
```

18 Troubleshooting

Symptom	Solution
POTCAR not found	Check `VASP_POTCAR_DIR` in `site.env` or GUI System Setup. Use the POTCAR chooser for unusual variants (`Ga_d`, `Fe_sv`).
`vasp_std` not found	Set the full binary path in `site.env` or GUI System Setup.
WARNING: may not have converged	Increase NSW (ionic) or NELM (electronic) in INCAR and re-run.
Wrong bands ($W = \Gamma$)	Conventional cubic cell detected; use the *primitive* cell for FCC/BCC so auto-detection picks cF/cI.
Projected DOS missing	Check instructions.txt: `projected on` and `projections on` are both accepted. Verify element names match POSCAR.
KPAR error	KPAR × NCORE must divide total MPI tasks. Use auto-defaults or set explicitly valid values.
GUI unreachable	Check SSH tunnel: `ssh -L 5001:localhost:5001 user@cluster`.
Project missing from dropdown	Folder must contain `POSCAR` + `settings.json` in the directory where `vasp-gui.py` was started.

19 Version History

v1.0 (2026-03-23) Initial release — GUI, workstation agent, PBEsol/R2SCAN/HSE06, SOC, GGA+U, convergence tests, Wannier90, phonopy.

v1.1 (2026-04–05) Per-task KPAR/NCORE smart defaults; SLURM agent with dependency-chained submission; cumulative DOS by element (no pymatgen); copy-from-relax/scf scripts.

v1.2 (2026-06) Collinear magnetism: MAGMOM per atom, spin-resolved band and DOS plots. Auto k-path detection (8 crystal systems). Fixed DOS projection parsing. Separate GUI and agent quick-reference sheets. Rewritten comprehensive LaTeX manual.

v1.3 (2026-06-15) Raw INCAR passthrough (`INCAR: ... END_INCAR`, global or per-step) for arbitrary VASP tags. Constant-pressure relaxation via `PRESSURE` (ISIF=3, IBRION=2, PSTRESS).

v1.4 (2026-06-19) High-throughput driver `ht-mp-scf.py`: Materials Project primitive cells → batch SCF + ELF, with per-material element/Z table and cross-material `afterok` chaining on SLURM.

Addendum: High-Throughput Quick Reference

Cheat-sheet for `ht-mp-scf.py` (full description in Section 5). For each Materials Project ID it downloads the **primitive** cell, stages an SCF calculation that also writes the electron localization function (`LELF = .TRUE.` → `ELFCAR`) on a medium `10 10 10` Γ-mesh (default; editable per material or via `--kmesh`), and builds a `runall.sh`.

```
pip install pymatgen mp-api
export MP_API_KEY=...              # Materials Project key
export VASP_POTCAR_DIR=...         # POTCAR library (used by the agent)
```

```
printf 'mp-149\nmp-2534\n' > highthrouput_list    # one mp-id per line

./ht-mp-scf.py --agent local  --mpi 16            # workstation,
    sequential
./ht-mp-scf.py --agent slurm  --profile slurm     # cluster,
    afterok-chained
./ht-mp-scf.py --agent slurm  --no-chain          # cluster,
    independent

./runall.sh                          # build inputs + run/submit each SCF
```

Output	Location
ELFCAR (electron localization)	`<mp-id>/02_scf/ELFCAR`
Staged inputs (POSCAR + instructions)	`_ht_inputs/<mp-id>/`
Element / Z table	`_ht_inputs/elements_Z.txt`

Note. **Transfer to a workstation:** only the project folder is needed (code, `modules/`, `profiles/`); VASP binaries and the POTCAR library already live there. `ht-mp-scf.py` needs internet for the MP download, so run it on the login node — `runall.sh` does none.

Note. **If the download step errors with `'dict' object has no attribute 'number'`:** that is an spglib / pymatgen version mismatch. Fix the root cause with `pip install -U spglib`. The driver also falls back to `Structure.get_primitive_structure()` automatically, so a run is never blocked by it.